

Ethical Hyper-Velocity (EHV): A Provably Deterministic Governance-Aware JIT Compiler Architecture for Agentic Systems

Riddhi Mohan Sharma, *Senior Member, IEEE*

IAM Architect & Non-Human Identity Governance

Enterprise Identity, M&A Integration, Regulated Environments

Email: riddhimohan@ieee.org

Abstract—As autonomous agentic systems scale across regulated critical infrastructures, the lack of mechanistic, hardware-rooted enforcement for high-frequency policy updates presents a fundamental safety gap. We introduce Ethical Hyper-Velocity (EHV), a novel architectural framework for the *formal verification of AI governance policies at runtime*. Unlike retrospective auditing frameworks (ISO/IEC 42001, NIST AI RMF) which introduce 14–30 day latencies, EHV relocates the Policy Enforcement Point (PEP) into the inference pipeline via a Governance-Aware Just-In-Time (JIT) Compiler. By integrating Conflict-free Replicated Data Types (CRDTs) for policy synchronization and Epoch-based Attestation Caching within Trusted Execution Environments (TEEs), EHV achieves *Sub-millisecond Formal Determinism (SMFD)*. We demonstrate via TLA+ formal verification that non-compliant agentic actions are computationally unreachable within the system’s bounded operating state space. We prove that $O(1)$ runtime enforcement can eliminate the traditional trade-off between deployment velocity and governance integrity, reducing Governance Latency from $O(\text{days})$ to $O(1)$.

Index Terms—AI Governance, Agentic Systems, Zero Trust Architecture, Formal Verification, TLA+, Trusted Execution Environments, Healthcare AI, Governance Latency

I. INTRODUCTION

Current AI governance relies on retrospective auditing and manual compliance gates. As autonomous agents—systems capable of multi-step reasoning and real-world action without human intervention—proliferate in regulated domains, this creates a *Governance Bottleneck*: model execution velocity exceeds oversight velocity by orders of magnitude.

Existing standards address the risk lifecycle (NIST AI RMF [1]) and management systems (ISO/IEC 42001 [2]), but they lack a *mechanistic enforcement model* at the execution layer. We argue that for autonomous agents, governance must move from a procedural gate to a verified system invariant.

We introduce **Ethical Hyper-Velocity (EHV)**, a framework for the **formal verification of AI governance policies at runtime**. EHV provides the first architecture for hardware-rooted, real-time policy enforcement that guarantees safety invariants even under high-frequency policy updates and distributed network conditions. By compiling governance into the inference stack, we transform policy from an external friction point into a provably deterministic architectural constraint.

A. Contributions

This paper makes the following contributions:

- 1) **The Governance Latency Problem**: We formalize the metric $GL = t_e - t_d$ and demonstrate its catastrophic implications in regulated autonomous systems.
- 2) **The Identity-Action Perimeter**: We extend NIST SP 800-207 ZTA from identity verification to agentic action verification, closing the “trusted identity, untrusted action” gap.
- 3) **Governance-Aware JIT Compiler**: We present an architecture that relocates the PEP into the inference pipeline using CRDTs and TEE-backed epoch caching.
- 4) **Formal Safety Proof**: We provide TLA+ specifications proving that non-compliant actions are computationally unreachable under all state-space interleavings.
- 5) **Threat Model**: We enumerate the attack surface and failure modes specific to governance-compiled agentic systems.

II. RELATED WORK

A. AI Governance Frameworks

The NIST AI Risk Management Framework [1] provides a lifecycle-oriented taxonomy (Map, Measure, Manage, Govern) but offers no enforcement mechanism; compliance is assessed retrospectively. ISO/IEC 42001 [2] establishes a management system for AI but inherits the PDCA audit cycle, yielding $GL \geq 14$ days. The EU AI Act [4] mandates risk classification and conformity assessment but delegates enforcement to national authorities operating on human timescales. None of these frameworks address real-time, pre-execution constraint enforcement for autonomous agents.

B. Zero Trust Architecture

NIST SP 800-207 [3] defines Zero Trust Architecture as continuous verification of the *subject* (identity) requesting access. In agentic contexts, this is necessary but insufficient: a cryptographically authenticated “Physician Twin” with valid credentials can still execute an action that violates a policy updated seconds ago. ZTA verifies *who*; EHV verifies *what*.

C. Formal Methods in Safety-Critical Systems

TLA+ and the TLC model checker [5] have been applied to distributed systems verification at Amazon Web Services [6]

and Microsoft Azure. Formal verification of AI safety constraints remains nascent. Amodei et al. [7] enumerate concrete AI safety problems but do not propose mechanistic enforcement. Constitutional AI [8] embeds behavioral constraints in training but provides no runtime guarantee. EHV bridges this gap by applying formal methods to the *governance enforcement layer* rather than the model itself.

D. Trusted Execution Environments

Intel SGX [9], AMD SEV-SNP [10], and ARM TrustZone provide hardware-rooted isolation for sensitive computation. Remote attestation protocols verify enclave integrity but introduce 200ms+ latency per attestation round-trip. EHV’s Epoch-based Attestation Caching amortizes this cost to $O(1)$ per inference call within an epoch.

E. CRDTs for Distributed State

Conflict-free Replicated Data Types [11] guarantee eventual consistency without coordination. Specifically, Join-Semilattice structures (LWW-Element-Sets) ensure monotonic policy convergence. EHV leverages this property to propagate safety constraints across partitioned networks without a central policy bottleneck.

F. Runtime Guardrail Systems

NVIDIA NeMo Guardrails [14] and Guardrails AI [15] provide runtime constraint enforcement for LLM outputs via programmable rules. These systems operate as software-layer filters between the model and the user. EHV differs in three fundamental dimensions: (1) *formal verification*—runtime guardrail systems offer no proof that unsafe outputs are unreachable; they filter probabilistically; (2) *hardware-rooted enforcement*—EHV’s PEP executes within a TEE, making bypass via process-level attacks infeasible; (3) *distributed policy synchronization*—guardrail systems assume centralized policy configuration, whereas EHV propagates constraints via CRDTs across partitioned networks. Runtime guardrails are necessary for general-purpose LLM deployment; EHV targets the stricter requirement of *provably deterministic* enforcement in regulated autonomous agents.

G. Emerging Agentic Security Standards

Recent work addresses runtime authorization specifically for multi-agent systems. Framework-layer middleware approaches provide identity passports and capability delegation for agent-to-agent communication but operate at the application layer, remaining vulnerable to process-level bypass. Policy compiler approaches express constraints as declarative rules over dynamic dependency graphs, enabling provenance-aware authorization. EHV complements these software-layer approaches by providing hardware-rooted enforcement at the inference layer, ensuring that even a compromised application runtime cannot circumvent governance constraints. A detailed comparison with specific emerging standards is deferred to a subsequent revision as the agentic security landscape matures.

III. PROBLEM FORMULATION: GOVERNANCE LATENCY

Definition 1 (Governance Latency). *For a policy decision event at time t_d and its enforcement at time t_e , Governance Latency is defined as:*

$$GL = t_e - t_d, \quad GL \in [0, \infty) \quad (1)$$

In traditional frameworks, GL spans 14–30 days due to manual review cycles. During this interval, an autonomous agent may execute N actions under a stale policy state:

$$N_{\text{unsafe}} = \lambda \cdot GL \quad (2)$$

where λ is the agent’s action throughput (actions/second). For illustrative healthcare parameters— $I = 5,000$ Physician Twin instances, $R = 100$ recommendations/hour—a 14-day GL yields:

$$\begin{aligned} N_{\text{unsafe}} &= I \cdot R \cdot 24 \cdot GL_{\text{days}} \\ &= 5,000 \times 100 \times 24 \times 14 = 168,000,000 \end{aligned} \quad (3)$$

EHV’s objective is to reduce GL to a constant bounded by TEE attestation overhead:

$$GL_{\text{EHV}} = O(1), \quad GL_{\text{EHV}} < 1\text{ms} \quad (4)$$

IV. SYSTEM ARCHITECTURE

The EHV architecture consists of three technical pillars.

A. Pillar 1: The Policy Compiler (CRDT-Based)

Policies are ingested as monotonic updates in a Join-Semilattice (LWW-Element-Set CRDT). Each policy update p_i carries a logical timestamp τ_i . The merge function guarantees:

$$\forall p_i, p_j : \tau_i > \tau_j \implies \text{merge}(p_i, p_j) = p_i \quad (5)$$

This ensures that even under network partitions, all agent nodes eventually converge on the most recent safety constraints without coordination overhead. The Global Ethical State S_G is defined as:

$$S_G = \bigsqcup_{i=1}^n S_i \quad (6)$$

where $\bigsqcup$ denotes the least upper bound in the semilattice and S_i is the local state at node i .

Clock Model Consideration. The current specification uses logical timestamps for conflict resolution. In production deployments with untrusted edge nodes, physical clock dependencies introduce NTP drift and timestamp injection risks. A hardened production variant would represent S_G as a directed acyclic graph (DAG) of cryptographically signed policy mutations resolved via vector clocks or explicit administrative authority hierarchies rather than raw timestamps. This is identified as a Phase 2 hardening target.

B. Pillar 2: Epoch-based Attestation Caching

Remote hardware attestation (Intel SGX EREPORT, AMD SEV-SNP) incurs 200ms+ latency per round-trip. EHV introduces *Policy Epochs*: the TEE validates the policy hash $H_p = \text{SHA-256}(S_G)$ once per epoch E_k . Within an epoch, the enforcement check reduces to:

$$\text{Verify}(a) = \begin{cases} O(1) & \text{if } H_p^{\text{local}} = H_p^{\text{epoch}} \\ \text{Re-attest} & \text{otherwise} \end{cases} \quad (7)$$

Epoch duration $|E_k|$ is configurable per domain. In the healthcare vertical, $|E_k| = 60\text{s}$ balances freshness against attestation cost.

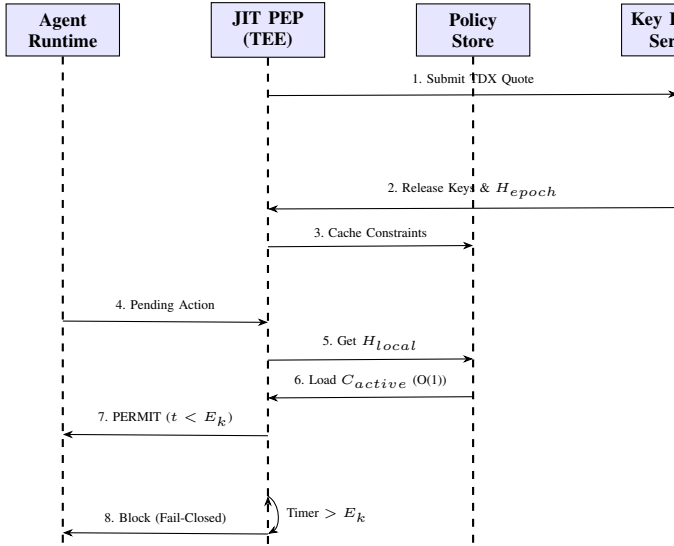


Fig. 1. The Core Decision Lifecycle. The JIT PEP restricts remote attestation to periodic epoch boundaries. If a network partition exceeds the duration, it structurally fails closed.

C. Pillar 3: The PEP in JIT

The Policy Enforcement Point is relocated from an external gateway to the token-generation layer of the inference pipeline. Before an agent emits an action a , the EHV JIT Compiler evaluates:

$$G(a, C) \in \{\text{PERMIT}, \text{DENY}, \text{ESCALATE}\} \quad (8)$$

where C is the current constraint set derived from S_G . DENY routes to a Safe Halt State. ESCALATE triggers a human-in-the-loop override for non-binary clinical judgment. The constraint set C is updatable per FDA PCCP [12] without agent redeployment.

D. Action Schema Extraction Layer (ASEL)

The PEP evaluates structured action representations, not raw token streams. EHV requires a pre-PEP *Action Schema Extraction Layer (ASEL)* that parses unstructured model outputs into typed action tuples ($\text{action_type}, \text{parameters}, \text{context}$). In the

healthcare vertical, ASEL maps clinical language to structured dosage, procedure, and referral schemas. For example, the output “administer 1.5mg/m² Vincristine IV” is parsed to $(\text{DOSAGE}, \{\text{drug} : \text{Vincristine}, \text{dose} : 1.5, \text{unit} : \text{mg/m}^2\}, \text{IV})$. The ASEL fidelity is domain-specific and is *not* formally verified within the current EHV specification. This is a scoped design boundary: the safety invariant I_g holds conditional on correct ASEL extraction. Formal verification of ASEL is identified as a primary target for future work.

V. FORMAL VERIFICATION (TLA+)

We specify the EHV system in TLA+ with four state variables:

- **PolicySet**: The current CRDT-merged policy state.
- **AgentAction**: The pending action under evaluation.
- **NetworkState**: $\in \{\text{CONNECTED}, \text{PARTITIONED}\}$.
- **EnforcementStatus**: $\in \{\text{PERMIT}, \text{DENY}, \text{ESCALATE}\}$.

A. Safety Invariant

$$I_g \triangleq \forall a \in \text{AgentAction} : \neg \text{Valid}(a, \text{PolicySet}) \implies \text{Status} = \text{DENY} \quad (9)$$

This states that no invalid action can reach a PERMIT state regardless of the system’s execution path.

B. Liveness Property

$$L \triangleq \forall p \in \text{PolicyUpdate} : \Diamond (\text{PolicySet}' = \text{merge}(\text{PolicySet}, p)) \quad (10)$$

Every policy update eventually propagates to all nodes (guaranteed by CRDT convergence).

C. Model Checking Results

Using the TLC Model Checker (v2026.05.04) with 10 parallel workers, covering all interleavings of asynchronous policy updates, network partitions, and concurrent agent actions:

- **States generated**: 1,738
- **Distinct states**: 324
- **State graph depth**: 8
- **Safety violations**: 0
- **Deadlocks**: 0
- **Temporal property violations**: 0 (5 branches checked)

The model was configured with $|\text{MaxPolicyVersion}| = 5$, $|\text{Actions}| = 3$ (safe_dosage , unsafe_dosage , escalate_case), covering the complete state space with a collision probability of 2.5×10^{-14} . The invariant I_g holds under all 324 distinct states: non-compliant actions are computationally unreachable within this configuration. The TLA+ specification and TLC output log are provided as supplementary artifacts.

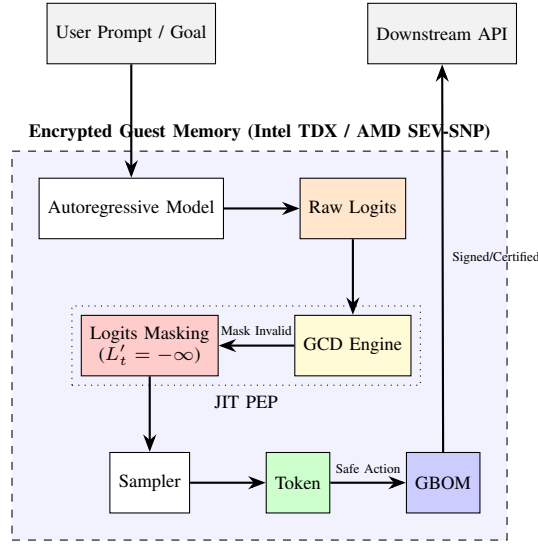


Fig. 2. System Architecture mapping the complete mediation pattern. The PEP intercepts unmasked logits inside encrypted guest memory, structurally eliminating the semantic parser gap.

D. Scope and Small-Model Considerations

The model checking results verify the enforcement logic for a bounded configuration with a single agent variable. Per the small-scope hypothesis [16], most design errors in concurrent and distributed systems manifest in models with small parameter values. However, we acknowledge that the current specification does not model concurrent multi-agent actions, realistic CRDT merge conflict scenarios with vector clocks, or unbounded policy version sequences. Extension to unbounded state spaces via inductive invariants using the TLA+ Proof System (TLAPS) is identified as primary future work. A proof-of-concept implementation demonstrating the enforcement pattern is available as a supplementary artifact.¹

VI. THREAT MODEL

We enumerate the attack surface specific to governance-compiled agentic systems.

A. Threat Categories

B. Trust Assumptions

- 1) The TEE hardware root of trust is uncompromised (Intel SGX, AMD SEV-SNP).
- 2) Policy updates are cryptographically signed by authorized issuers.
- 3) The PEP binary within the TEE is measured and attested at epoch boundaries.
- 4) Network partitions are eventually resolved (partial synchrony model).

C. Failure Mode: Non-TEE Environments

In environments lacking Confidential Computing support, SMFD degrades. The system falls back to out-of-band audit with $GL > 0$, governed by NIST SP 800-53 SI-17 fail-safe provisions [13]. This is the primary architectural friction point.

TABLE I
EHV THREAT MODEL

ID	Threat	Mitigation	Residual
T1	Policy poisoning via CRDT injection	Authenticated updates; cryptographic signing of p_i	Low
T2	TEE side-channel (Spectre/Foreshadow)	Epoch rotation; attestation refresh	Medium
T3	Stale epoch exploitation	Configurable $ E_k $; forced re-attestation on critical updates	Low
T4	Network partition > epoch duration	CRDT convergence; fail-closed architecture under partition	Low
T5	Model obfuscation (adversarial encoding)	PEP operates on decoded action semantics	Medium
T6	Governance bypass via prompt injection	PEP is post-generation; operates on output, not input	Low
T7	Clock-skew / NTP poisoning on edge nodes	Logical timestamps; signed policy DAG (future)	Medium

¹<https://github.com/riddhimohansharma/ehv-runtime>

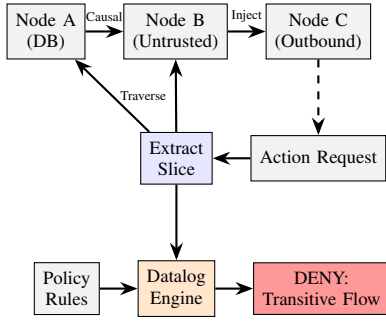


Fig. 3. Multi-Agent Causal Provenance. The Reference Monitor extracts a backward slice of the state dependency graph to evaluate transitive exploits like indirect prompt injection via Datalog rules.

D. Epoch Staleness Window (ESW) Analysis

Within a policy epoch E_k , a critical policy update arriving at time t_u is not enforced until the next epoch boundary. The maximum staleness window is:

$$ESW_{\max} = |E_k| - 1 \quad (11)$$

During this window, actions are governed by the previous policy version. For healthcare parameters ($\lambda = 500,000$ actions/hour aggregate, $|E_k| = 60s$):

$$\begin{aligned} N_{\text{stale}} &\leq \lambda \cdot ESW_{\max} \\ &= \frac{500,000}{3,600} \times 59 \approx 8,194 \text{ actions} \end{aligned} \quad (12)$$

Compared to the legacy $N_{\text{unsafe}} = 168,000,000$ under 14-day GL , this represents a **5 order-of-magnitude improvement**. For ultra-critical updates (e.g., drug withdrawal), EHV supports forced mid-epoch re-attestation via an EMERGENCY_EPOCH_RESET signal, reducing ESW to network propagation latency ($< 1s$).

E. Fail-Closed Partition Semantics

When a network partition persists beyond the epoch boundary $|E_k|$ and the local node cannot reach the attestation authority, EHV enforces strict fail-closed semantics:

$$t - t_{\text{last_attest}} > |E_k| \implies G(a, C) = \text{DENY} \quad \forall a \quad (13)$$

The JIT PEP transitions to a Safe Halt State, blocking all outgoing tool and action executions until attestation is restored. This guarantees that no agent can execute unconstrained actions indefinitely under a stale policy state, even during extended network outages.

VII. CASE STUDY: PEDIATRIC ONCOLOGY DOSAGE

Consider an FDA-mandated reduction in Vincristine dosage from 1.5 mg/m^2 to 0.75 mg/m^2 due to a new neurotoxicity signal.

A. Legacy System ($GL = 14 \text{ days}$)

Manual protocol review, committee approval, EHR update, and staff retraining yield $GL = 14$ days. During this interval, across 5,000 Physician Twin instances processing 100 recommendations/hour:

$$\begin{aligned} N_{\text{toxic}} &= 5,000 \times 100 \times 24 \times 14 \\ &= 168,000,000 \text{ unconstrained actions} \end{aligned} \quad (14)$$

Even at a 0.03% violation rate, this produces 50,400 potentially toxic dosage recommendations.

B. EHV System ($GL < 1ms$)

The dosage update propagates via CRDT in < 1 second. The EHV JIT Compiler intercepts any model output exceeding the new 0.75 mg/m^2 limit. The action is DENY'd at the hardware level, regardless of the underlying LLM's training data. The GBOM (Governance Bill of Materials) provides a cryptographic receipt binding each clinical recommendation to the policy version that governed it.

VIII. DISCUSSION

A. The Velocity-Ethics Co-Production Principle

In EHV-compliant systems, deployment velocity V and governance integrity I are positively correlated:

$$\frac{\partial V}{\partial I} \geq 0 \quad (\text{EHV}) \quad \text{vs.} \quad \frac{\partial V}{\partial I} < 0 \quad (\text{traditional}) \quad (15)$$

The causal mechanism underlying this sign reversal is *pre-clearance elimination of post-hoc audit*. In traditional architectures, governance imposes friction as a linear cost: $V = V_0 - k \cdot I$, where k represents audit overhead per unit of integrity. In EHV, because compliance is verified at inference time, no post-deployment audit backlog accumulates. The deployment pipeline is never blocked by governance review queues. Formally: $V_{\text{EHV}} = V_0 \cdot f(I)$ where $f(I) \geq 1$ when pre-execution verification eliminates the need for retrospective compliance gates. Governance becomes the mechanism of acceleration rather than friction.

B. The GBOM for M&A Due Diligence

EHV introduces the **Governance Bill of Materials (GBOM)**: a cryptographic audit trail that binds each autonomous decision to the specific policy version, TEE attestation epoch, and enforcement outcome that governed it. This enables acquirers to verify the governance posture of an AI stack with the same rigor applied to financial audits.

C. Limitations

- 1) **TEE dependency**: SMFD requires Confidential Computing hardware. Non-TEE deployments degrade to out-of-band auditing.
- 2) **Epoch granularity**: The tradeoff between freshness and attestation cost requires domain-specific tuning.

- 3) **Semantic gap:** The PEP operates on structured action representations; unstructured model outputs require a semantic parsing layer whose fidelity is not formally verified. In future iterations, this semantic gap will be closed by integrating a GPU-accelerated token-masking logits processor (Grammar-Constrained Decoding) directly into the JIT compiler, replacing the unverified parsing layer with hardware-enforced grammar constraints.
- 4) **Bounded model checking:** The TLA+ verification covers the complete state space for a bounded configuration (depth 8, 324 distinct states) per the small-scope hypothesis [16]. Extension to unbounded state spaces via inductive invariants (TLAPS) is identified as primary future work.

IX. CONCLUSION

EHV transforms governance from a manual gate into a hardware-rooted system invariant. By compiling policy enforcement into the inference pipeline via a Governance-Aware JIT Compiler, backed by CRDT-synchronized policy state and TEE-anchored attestation caching, EHV eliminates Governance Latency for agentic systems in regulated domains.

The formal verification demonstrates that non-compliant actions are computationally unreachable within the system’s state space. The Velocity-Ethics Co-Production Principle establishes that governance integrity and deployment speed are not adversarial but co-productive when enforcement is architectural rather than procedural.

Future work includes: (1) extending the TLA+ specification to unbounded state spaces via inductive invariants, (2) benchmarking SMFD latency on production TEE hardware (Intel TDX, AMD SEV-SNP) with measured PEP overhead per inference call, (3) formal verification of the Action Schema Extraction Layer (ASEL) for healthcare-specific clinical language, (4) CRDT propagation latency characterization under realistic WAN conditions, and (5) developing the FAITH (Formal Attestation of Identity-Trust Hierarchies) framework for cryptographic binding of digital twin credentials to EHV audit trails.

AI TOOL DISCLOSURE

AI tools (Claude, Gemini) were used for prose refinement, LaTeX formatting, and literature survey assistance. All technical claims, formal specifications (TLA+), architectural design decisions, mathematical derivations, and the system architecture are the original intellectual work of the author. The TLA+ specification was authored and independently verified by the researcher using the TLC model checker. The proof-of-concept implementation was developed by the author.

ACKNOWLEDGMENTS

The author acknowledges the Cloud Security Alliance for technical review guidance and the India AI Impact Summit 2026 for providing the forum where DPI-native governance patterns informed the EHV architecture.

REFERENCES

- [1] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023.
- [2] International Organization for Standardization, “ISO/IEC 42001:2023 — Artificial intelligence — Management system,” 2023.
- [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST SP 800-207, Aug. 2020.
- [4] European Parliament, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act),” June 2024.
- [5] L. Lamport, “Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers,” Addison-Wesley, 2002.
- [6] C. Newcombe, T. Rath, F. Zhang, B. Murat, and M. Brooker, “How Amazon Web Services Uses Formal Methods,” *Commun. ACM*, vol. 58, no. 4, pp. 66–73, 2015.
- [7] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete Problems in AI Safety,” arXiv:1606.06565, 2016.
- [8] Y. Bai et al., “Constitutional AI: Harmlessness from AI Feedback,” arXiv:2212.08073, 2022.
- [9] V. Costan and S. Devadas, “Intel SGX Explained,” *IACR Cryptology ePrint Archive*, 2016.
- [10] AMD, “AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More,” AMD White Paper, 2020.
- [11] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free Replicated Data Types,” in *Proc. SSS 2011*, LNCS, vol. 6976, pp. 386–400, 2011.
- [12] U.S. Food and Drug Administration, “Marketing Submission Recommendations for a Predetermined Change Control Plan for AI/ML-Enabled Device Software Functions,” FDA Guidance, 2023.
- [13] Joint Task Force, “Security and Privacy Controls for Information Systems and Organizations,” NIST SP 800-53 Rev. 5, Sept. 2020.
- [14] NVIDIA, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications,” NVIDIA Technical Report, 2023. <https://github.com/NVIDIA/NeMo-Guardrails>
- [15] Guardrails AI, “Guardrails: Adding Guardrails to Large Language Models,” GitHub Repository and Documentation, 2023. <https://github.com/guardrails-ai/guardrails>
- [16] D. Jackson, “Software Abstractions: Logic, Language, and Analysis,” MIT Press, 2006.